Instant Payments Namibia

P2P Payment Integration Challenge

Technical Integration Submission

Project repo: https://github.com/alien-code-hunter/Instant-Payments-Namibia-Challenge

Project	Instant Payments Namibia – P2P Payment Application
Submission Type	Developer Integration Challenge
Technology Stack	React, Node.js, Express
Date	14 March 2026
Prepared by	Meduletu Tangeni Kamati
Email	Meduletu.kamati@gmail.com

1. Introduction

This submission presents a web-based Person-to-Person (P2P) payment application developed for the Instant Payments Namibia (IPN) Developer Integration Challenge. The application demonstrates integration with the specified mock payment API by allowing users to initiate P2P transactions through a React-based payment form, which submits validated payment data to a Node.js and Express backend endpoint (POST /api/p2p-payment). The system automatically generates a unique clientReference, performs both client-side and server-side validation in accordance with the API specification, and displays transaction responses including status, transaction ID, error codes, and descriptive messages. The implementation focuses on correctness, clear validation handling, and compliance with the integration requirements while providing a simple and user-friendly interface for testing payment transactions.

2. Application Screenshots

The following screenshots are tests of the functionality of the application.

2.1. Payment Form

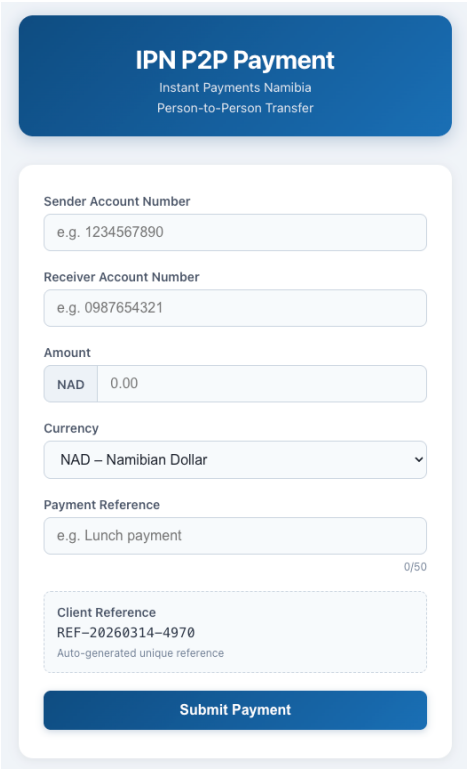

Image 1: The photo of the application form without filled values

2.2. Transaction Result – Successful Payment

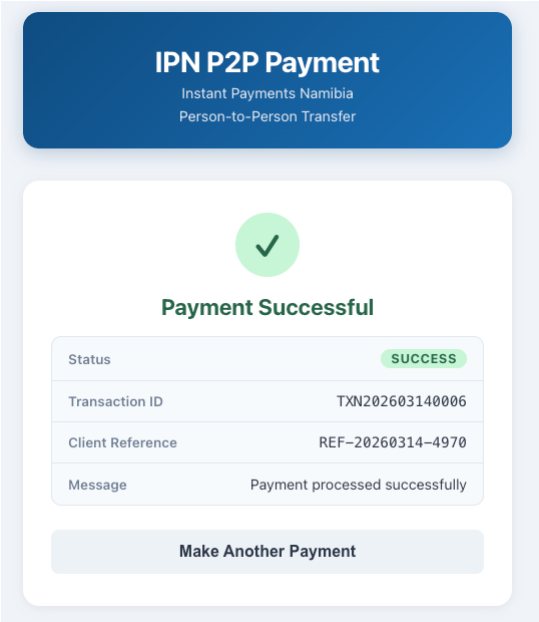

Image 2: Successful payment of the application

2.3. Transaction Result – Failed Payment

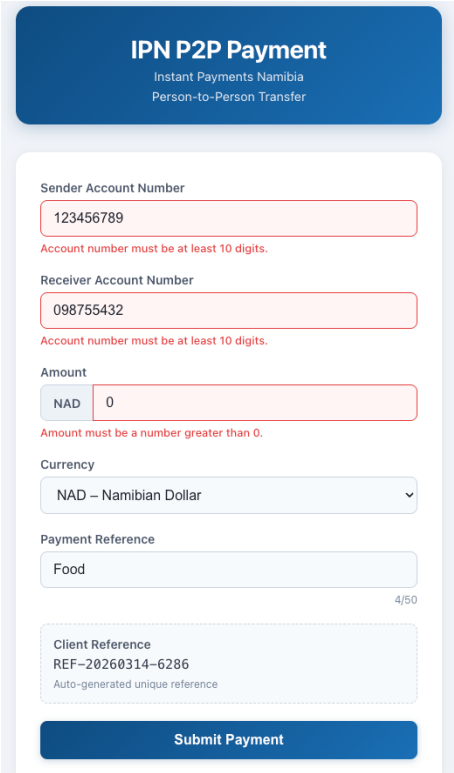

Image 3: Unsuccessful payment due to violation of at least 1 rule

2.4. Validation Errors

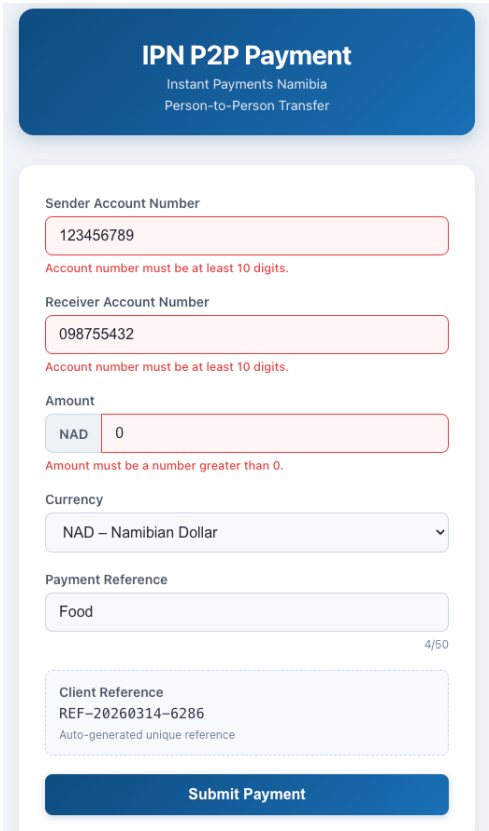

Image 4: All validation rules enforced

3. Technology Stack

Component	Technology
Frontend	React
Backend	Node.js with Express
API Type	Mock API

4. Setup Instructions

4.1. Prerequisites:

- Install Node.js 18 or higher
- npm (Node Package Manager)
  - *nvm install node*
  - *nvm use 18*
- Verifying installation (expected result: v18.x.x and 9.x.x respectively)
  - node -v
  - npm -v

4.2. Clone the Project Repository

git clone https://github.com/alien-code-hunter/Instant-Payments-Namibia-Challenge
cd Instant-Payments-Namibia-Challenge

4.3. Starting the Backend:

cd backend
npm install
npm start

4.4. Starting Frontend (new terminal):

cd frontend
npm install
npm start

4.5. Running Frontend Tests:

cd frontend
npm test

5. Project Structure

```
├── backend/
│   ├── server.js          # Express mock API server
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── App.js         # Main React component (form + result display)
│   │   ├── App.css        # Styling
│   │   └── App.test.js    # Component tests
│   └── package.json
└── README.md
```

Note: Other project files and resources such as images and favicons are not included in this structure.

6. Integration Implementation

API Endpoint: POST /api/p2p-payment. The backend validates all fields and returns a JSON response containing status, errorCode, transactionId, and message.

Frontend Flow:

1. User enters sender account, receiver account, amount, currency, and reference.
2. A clientReference is automatically generated in the format REF-YYYYMMDD-XXXX.
3. Client-side validation runs before submission.
4. If validation passes, the request is sent to /api/p2p-payment.
5. The API response is displayed in a result card.

7. Validation Rules

Field	Data Type	Required	Validation Rules
clientReference	String	Yes	Auto-generated unique reference
senderAccountNumber	String	Yes	Digits only, minimum 10 characters
receiverAccountNumber	String	Yes	Digits only, minimum 10 characters
amount	Number	Yes	Must be greater than 0
currency	String	Yes	Must be NAD
reference	String	Yes	Maximum 50 characters

8. Testing

- All form fields use correct data types.
- Validation rules are enforced on both client and server.
- Invalid inputs trigger appropriate validation messages.
- API responses are processed and displayed correctly.
- Edge cases such as invalid input and duplicate references are handled.

9. Assumptions

- Authentication is not required for this integration challenge.
- No database is used; client references are tracked in memory.
- Only NAD currency is supported.
- clientReference is automatically generated by the frontend.
- The mock API always succeeds for valid requests.
- CORS is enabled for all origins during development.
- The API URL defaults to http://localhost:3001 and can be overridden using REACT_APP_API_URL.